PlayCarys

Application Workflows

What each role can actually do, traced through real routes and pages — not the launch/deploy steps.

Every flow below is grounded in a real controller route or a real front-end page in this project — nothing here is a guess at intended behavior. Where a flow is incomplete (a page with no backend route, or a route with no page), it is marked GAP rather than smoothed over.

1. Player Workflow

Sign-up & sign-in

1. Open the site (index.html) — Sign In panel, or continue as guest.
2. Register: choose Player, pick a username/email/password.
3. Or sign in: email + password. A conflicting active session on another device triggers a handoff request (approve/deny from the other device, or a 15s timeout).

`POST /auth/register` · `POST /auth/login` · `POST /auth/forgot-password` · `POST /auth/reset-password`

Browsing & playing a game

1. Dashboard (player/dashboard.html) — stats, recent activity, quick links.
2. Games (player/games.html) — browse the catalog, filter by genre.
3. Play (player/play.html) — launches the selected game; on finish, submits a result.

`GET /games` · `GET /games/:id`

> *GAP — Solo play sessions use a client-generated ID ("SOLO"+timestamp) that is never registered with the backend via POST /sessions — there is currently no POST route on GameSessionController at all (only GET, GET by id, GET by player, and PATCH :id/end exist). Match results and the resulting PCP/XP award are recorded through a different, local-first path, not this session lifecycle.*

Match history & leaderboard

- player/match-history.html — past results
- player/match-details.html — one match, expanded
- player/leaderboard.html — platform rankings

`GET /player-profiles/:userId/match-history` · `GET /player-profiles/leaderboard` · `GET /match-results/player/:playerId`

Tournaments

1. Browse (player/tournaments.html) and register for an open tournament.
2. Track entries in player/my-tournaments.html.
3. Chat with other entrants and fetch a per-match token — both genuinely wired and working from player/tournaments.html.

`POST /tournaments/:id/register` · `GET /tournaments/:id/chat` · `GET /tournaments/matches/:matchId/token`

> *GAP — Everything past the token — viewing the bracket, reporting/confirming/disputing a result, and the organizer resolving a dispute — has a real backend route but no caller anywhere in the frontend (checked tournaments.html and publish-tournament.html specifically). A player can register, chat, and get a match token, but there is currently no UI path to actually report how a tournament match went.*

Notifications & profile

- player/notifications.html — broadcasts, direct messages, replies, mark read, clear
- player/profile.html — edit username/email/phone/avatar/password

`GET /notifications` · `PATCH /notifications/:id/read` · `POST /notifications/:id/reply`

Social — friends & DMs

The backend has a complete SocialModule: player search, friend requests (send/accept/reject), a friends list, and direct messages. There is also a complete, self-contained frontend module for it: js/social.js — a real, backend-driven API layer plus a full draggable floating-chat-widget UI (search, friends list, requests, conversation view), written specifically because most of this app syncs through one big local-DB sync instead of real per-action calls, and chat is the one feature that breaks hardest

under that pattern.

`GET /players/search` · `POST /friends/request` · `POST /friends/requests/:id/respond` · `GET/POST /messages`

> *GAP — js/social.js is never <script>-included by any page — checked every pages/*.html and index.html. The module is finished and correct on both ends; it is simply never loaded, so no button anywhere on the site can open it. This is a one-line fix (add the <script> tag to whichever pages should offer it) away from being a real feature, not a missing one.*

2. Publisher Workflow

Sign-up & profile

Register choosing Publisher; publisher/profile.html for self-service edits (studio name, email, password).

Game lifecycle — the core publisher flow

1. Create a listing (publisher/publish.html) — name, genre, description.
2. Upload the build as a .zip (publisher/mygames.html -> game-review) — scanned automatically for risky patterns (storage access, eval, dynamic scripts, external origins) before it can be approved.
3. Wait for review — an admin/superuser approves or rejects (see Admin Workflow below).
4. If approved: toggle enabled/disabled at will, subject to R-GAME-4 (can't enable an unapproved build).
5. If rejected: edit the listing — this now automatically resubmits it to 'pending' (fixed this session; there was previously no path back from 'rejected' at all).
6. Delete own game — ownership-checked, separate from the admin/superuser delete path.

```
POST /games · POST /games/:id/bundle (multipart, 50MB cap) · PATCH /games/:id · PATCH /games/:id/state
DELETE /games/:id/self?publisher_id=... · GET /games/:id/scan-manifest · GET /games/:id/review
```

Publishing a tournament

publisher/publish-tournament.html — create a tournament for one of their games, then start it once registration is complete (moves it into the match-lifecycle routes under tournament-play).

```
POST /tournaments · POST /tournaments/:id/start
```

Game saves

A per-player cloud-save API — player/play.html genuinely calls both ends of it (load on start, save on progress). Fully wired, not a gap.

```
GET /games/:id/save · PUT /games/:id/save
```

3. Moderator Workflow

Report triage — the main loop

1. Reports list (moderator/reports.html) — pending, under-review, resolved, rejected.
2. Review -> marks pending as under-review.
3. Take Action -> opens the decision panel: severity, notes.
4. Resolve -> for a real player report, logs a formal moderator action AND resolves the report in one call; for a direct-contact message (no player on the other end), just a status update.
5. Dismiss -> rejects the report with notes.
6. Warn -> sends a warning without closing the report (kept open deliberately).
7. Escalate to Ban -> hands off to the Bans screen.

`PATCH /reports/:id/status` · `POST /moderator-actions/act-on-report` · `POST /moderator-actions/handle-violation`

> *GAP — PATCH /reports/:id/status carries no @Roles guard on the backend — every other write route in this project is staff-only, but this one is currently reachable by anyone who can reach the API at all, not just moderators. Worth tightening.*

Bans

moderator/bans.html — issue a ban (duration + reason -> sets user.status, player_profile status, and the ban record server-side, with a duplicate-ban guard), or lift one.

`POST /moderator-actions/handle-violation (action_type: ban)` · `PATCH /moderator-actions/bans/:playerId/lift`

Live sessions

moderator/sessions.html — view active game sessions platform-wide (read-only oversight).

`GET /sessions`

4. Admin Workflow

Game review — the gatekeeping step every publisher build passes through

1. Approvals queue (admin/approvals.html) lists pending builds.
2. Game Review (admin/game-review.html) shows the scan report (severity-ranked findings) and the listing details.
3. Approve or Reject, with an optional note — approving a build the scanner flagged 'fail' requires a written justification.
4. A decision notifies the publisher and (for non-superuser admins) escalates a copy to superuser.

`PATCH /games/:id/approve` · `GET /games/:id/scan-manifest`

Staff account management

admin/users.html — role management for the platform's regular (player/publisher) accounts: edit, suspend, reinstate, ban, ban+delete.

`PATCH /users/:id` · `PATCH /users/:id/status` · `DELETE /users/:id`

> *GAP — Time-limited suspension ('suspend for N days') is tracked only client-side (a suspendUntil field) — the backend's UpdateUserStatusDto only knows active/banned/suspended, with no expiry concept and no auto-reinstate job. A suspension currently never lifts itself.*

Dashboard & escalations

admin/dashboard.html and admin/hub.html surface platform activity and route escalations up from moderators.

5. Superuser Workflow

Superuser inherits every admin and moderator capability (RolesGuard's superuser check bypasses every @Roles(...) restriction platform-wide) plus:

Staff account creation — the one thing admins can't do

superuser/admins.html — create new admin/moderator accounts, suspend/reinstate them with a reason, all through the same signed-token path.

`POST /users` · `PATCH /users/:id/status`

Full user directory

superuser/users.html — every account on the platform, any role, with quick suspend/unban/ban actions.

Game moderation without ownership

superuser/games.html — blacklist (force-disable) or delete ANY game, not just ones staff themselves own — the specific gap this session's Gap-6 work filled (games.service.ts moderateState()), since the publisher-facing endpoints are ownership-locked and previously had no staff equivalent.

`PATCH /games/:id/moderate` · `DELETE /games/:id`

Tournament administration

Create, edit, or delete any tournament (not the player-facing register/unregister — genuine administration).

`POST /tournaments` · `PATCH /tournaments/:id` · `DELETE /tournaments/:id`

Platform settings & reports

- superuser/settings.html — platform-wide configuration
- superuser/reports.html, superuser/hub.html, superuser/dashboard.html — oversight views

6. Cross-cutting: the tournament match lifecycle

This is the one workflow that spans three roles in sequence, worth tracing end to end — and it is where the gap between "backend built" and "reachable from the UI" is sharpest:

1. Publisher creates the tournament (POST /tournaments), players register (POST /tournaments/:id/register). — wired.
2. Publisher starts it (POST /tournaments/:id/start) — this generates the bracket. — no caller found.
3. Players chat and fetch a per-match token (GET /tournaments/:id/chat, GET /tournaments/matches/:matchId/token). — wired, from player/tournaments.html.
4. A player reports the result (POST /tournaments/matches/:matchId/report); the opponent confirms (…/confirm) or disputes (…/dispute). — no caller found.
5. A disputed match surfaces to the organizer's dispute queue (GET /tournaments/disputes/:organizerId) for manual resolve (…/resolve). — no caller found.

> *GAP — Registration, chat, and token issuance work end to end. Starting a tournament, viewing its bracket, and the entire report -> confirm/dispute -> resolve chain do not have a UI path yet, even though all of it exists and is implemented in tournament-play.controller.ts (13 routes total). A tournament can currently be entered and chatted in, but not actually played through to a recorded result via the UI.*

7. Summary — what exists vs. what's reachable

Fully wired (backend + real UI page calling it):

- Auth (login/register/reset)
- Game browsing, upload, review/approval, state toggle
- Reports -> moderator actions -> bans
- Staff user management (admin/superuser)
- Notifications (read/reply/clear)
- Match history & leaderboard (read)
- Tournament registration, chat, and per-match token issuance
- Per-game cloud save (GET/PUT /games/:id/save)

Backend built, no UI wired yet (GAP):

- Social — js/social.js is a complete, working module, just never <script>-included anywhere
- Tournament match play — start / bracket / report / confirm / dispute / resolve

Known process gaps (flagged, not fixed, this pass):

- PATCH /reports/:id/status has no role guard
- Suspensions have no expiry/auto-reinstate
- Solo play sessions never register with the backend session lifecycle